

Modularization in PL/SQL

summarize this

Motivation for Functions and Procedures: Modular Code.

Modularization is the process by

which you break up large blocks of code into smaller pieces (modules) that can be called

by other modules. Modularization of code is analogous to normalization of data, with

many of the same benefits and a few additional advantages. With modularization, your code becomes:

- **More reusable.** By breaking up a large program or entire application into individual components that plug-and-play together, you will usually find that many modules are used by more than one other program in your current application. Designed properly, these utility programs could even be of use in other applications!
- **More manageable.** Which would you rather debug: a 1,000-line program or five individual 200-line programs that call each other as needed? Our minds work better when we can focus on smaller tasks. You can also test and debug on a per-program scale (called unit testing) before individual modules are combined for a more complicated integration test.
- **More readable.** Modules have names, and names describe behavior. The more you move or hide your code behind a programmatic interface, the easier it is to read and understand what that program is doing. Modularization helps you focus on the big picture rather than on the individual executable statements. You might even end up with that most elusive kind of software: self-documenting code.
- **More reliable.** The code you produce will have fewer errors. The errors you do find

will be easier to fix because they will be isolated within a module. In addition, your code will be easier to maintain because there is less of it and it is more readable.

Forms of Modularization: PL/SQL offers the following structures that modularize your code in different ways:

43

Dr. Raihan Kamal

CHAPTER 6. FUNCTIONS AND PROCEDURES

- **Procedure.** A program that performs one or more actions and is called as an executable PL/SQL statement. You can pass information into and out of a procedure through its parameter list.
- **Function.** A program that returns data through its RETURN clause, and is used just like a PL/SQL expression. You can pass information into a function through its parameter list.
- **Database trigger.** A set of commands that are triggered to execute (e.g., log in, modify a row in a table, execute a DDL statement) when an event occurs in the database.
- **Package.** A named collection of procedures, functions, types, and variables. A package is not really a module (it's more of a meta-module), but it is so closely related that.

Summary:

Modularization in programming involves breaking large code blocks into smaller, reusable modules (similar to how data normalization works). This approach offers several benefits:

- **Reusability:** Modules can be reused across multiple programs or even different applications.
- **Manageability:** Smaller modules are easier to debug and test individually (unit testing).
- **Readability:** Named modules make code easier to understand and maintain.
- **Reliability:** Modular code tends to have fewer and more easily isolated errors.

PL/SQL provides the following modularization structures:

- **Procedure:** Executes actions and can pass data via parameters.
- **Function:** Returns a value and can also accept parameters.
- **Database Trigger:** Automatically runs code in response to database events.
- **Package:** A collection of procedures, functions, and variables grouped together (a meta-module).

explain me all the differences between functions and procedures

Here is a detailed comparison of **functions** and **procedures** in **PL/SQL**, showing all the key differences:

◆ 1. Purpose

Aspect	Function	Procedure
Main Use	Returns a single value	Performs an action or task (may or may not return data)

◆ 2. Return Value

Aspect	Function	Procedure
Return	Must return a value using the <code>RETURN</code> statement	May return values via <code>OUT</code> or <code>IN OUT</code> parameters, not required to return anything

◆ 3. Invocation Method

Aspect	Function	Procedure
How It's Called	Can be called within SQL expressions	Must be called as a PL/SQL statement , not in SQL expressions

Example:

```
sql

-- Function in SQL:
SELECT get_discount_price(100, 'Silver') FROM dual;

-- Procedure in PL/SQL:
BEGIN
    apply_discount(100, 'Silver', :discounted_price);
END;
```

◆ 4. Use in SQL Statements

Aspect	Function	Procedure
SQL Use	Can be used in SQL queries (<code>SELECT</code> , <code>WHERE</code> , etc.)	Cannot be used directly in SQL statements

◆ 5. Parameters

Aspect	Function	Procedure
Parameters	Accepts <code>IN</code> parameters only	Can use <code>IN</code> , <code>OUT</code> , and <code>IN OUT</code> parameters

◆ 6. Syntax

| Function Example |

```
sql
```

```
CREATE FUNCTION get_tax(price NUMBER)
RETURN NUMBER
IS
BEGIN
    RETURN price * 0.15;
END;
```

| Procedure Example |

sql

```
CREATE PROCEDURE update_salary(emp_id NUMBER, new_sal NUMBER)
IS
BEGIN
    UPDATE employees SET salary = new_sal WHERE id = emp_id;
END;
```

◆ 7. Transaction Control

Aspect	Function	Procedure
COMMIT/ROLLBACK	Not recommended (can cause errors)	Allowed (but still discouraged in best practice)

◆ 8. When to Use

Use Case	Function	Procedure
When a result is needed	✓ Yes	✗ Not ideal
When an action is needed	✗ Not ideal	✓ Yes
Inside SQL queries	✓ Can be used	✗ Cannot be used
Complex logic with multiple outputs	✗ Not ideal	✓ Best suited

✓ Summary Table

Feature	Function	Procedure
Returns a value	Yes (via <code>RETURN</code>)	No (or via <code>OUT</code> params)
Used in SQL	Yes	No
Parameter modes	IN only	IN, OUT, IN OUT
Call type	Part of expression	Standalone PL/SQL statement
Suitable for logic	Single value computation	Complex business logic
COMMIT/ROLLBACK	Not recommended	Allowed

Let me know if you want real examples or a quiz to test your understanding!

can functions update a table and can procedure?
give me the reasons and examples when it will not work

Great question! Here's a clear explanation:

◆ Can Functions and Procedures Update a Table in PL/SQL?

Operation	Function	Procedure
Update a table	⚠ Yes, but not recommended	✓ Yes, fully allowed

✓ Procedures – Can Safely Update Tables

Procedures are designed to **perform actions**, such as inserting, updating, or deleting data. They can freely execute **DML (Data Manipulation Language)** statements like:

- `INSERT`
- `UPDATE`
- `DELETE`

♦ Example: Procedure that updates a table

sql

```
CREATE OR REPLACE PROCEDURE update_salary_proc(emp_id NUMBER, new_sal NUMBER) IS
BEGIN
  UPDATE employees SET salary = new_sal WHERE id = emp_id;
END;
```

✓ This will work perfectly when called in PL/SQL:

sql

```
BEGIN
  update_salary_proc(101, 60000);
END;
```

⚠ Functions – Technically Can Update, But With Restrictions

A function **can** update a table **if** it's called **from PL/SQL**, **not** from **SQL statements** like `SELECT`.

PL/SQL allows side effects in functions (like DML), but **SQL does not**.

● **Function will fail if used in SQL and tries DML**

◆ Example: Function that updates a table

sql

```
CREATE OR REPLACE FUNCTION update_bonus_func(emp_id NUMBER, bonus NUMBER)
RETURN NUMBER IS
BEGIN
    UPDATE employees SET bonus_amt = bonus WHERE id = emp_id;
    RETURN bonus;
END;
```

This function **works fine** when called in PL/SQL block:

sql

```
BEGIN
    update_bonus_func(101, 5000);
END;
```

✗ But this will cause an error if called inside SQL:

sql

```
SELECT update_bonus_func(101, 5000) FROM dual;
```

● **Error:**

pgsql

ORA-14551: cannot perform a DML operation inside a query

🔍 Why does it fail in SQL?

- SQL queries must be side-effect free.
- Oracle prevents functions that modify data from being used inside SQL, because:
 - SQL expects pure expressions.

- DML inside SQL can cause **unpredictable behavior, data integrity issues, or mutating table errors.**

✅ When to Use What:

Situation	Use Function?	Use Procedure?
Return a calculated value	✅ Yes	❌ Not preferred
Perform a table update	⚠️ Only in PL/SQL	✅ Yes
Call from SQL	❌ No DML inside	✅ Yes (via PL/SQL)

🔑 Summary

- ✅ **Procedures:** Safely update tables in all cases.
- ⚠️ **Functions:** Can update only when used **in PL/SQL**, not in SQL queries.
- ❌ **Calling a function with DML inside a SQL statement will cause an error.**

Let me know if you want me to show examples with `INSERT` , `DELETE` , or inside a trigger too!

Give me theoretical tricky question and answers about procedures

Here are some **theoretical and tricky PL/SQL procedure-related questions** that test deep understanding, not just definitions:

♦ Q1. Can a procedure call itself recursively in PL/SQL?

Answer:

- ✅ **Yes**, a procedure can call itself recursively. This is useful for tasks like traversing trees or

performing repetitive breakdowns. However, it must be written carefully to avoid infinite loops or exceeding the call stack.

Example use case: Calculating factorials or processing hierarchical data.

♦ Q2. Can a procedure be created without any parameters?

Answer:

✓ **Yes**, procedures can have **zero or many parameters**. Parameters are optional and only needed when data must be passed into or out of the procedure.

Example:

```
sql

CREATE OR REPLACE PROCEDURE hello_world IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello!');
END;
```

♦ Q3. What happens if you try to use a `RETURN` statement in a procedure?

Answer:

✓ It is **allowed**, but only to **exit the procedure early** — not to return a value.

Unlike functions, procedures do **not** use `RETURN` to send a result back.

So this is valid:

```
sql

IF some_condition THEN
  RETURN; -- exit procedure
END IF;
```

But this is invalid in a procedure:

sql

`RETURN 5; --` ❌ *Error: cannot return a value from procedure*

♦ Q4. Can a procedure be used in a SQL `SELECT` statement?

Answer:

❌ No, procedures cannot be used directly in SQL statements like `SELECT`, `WHERE`, or `FROM`. Only **functions** can be used in SQL expressions.

This will raise an error:

sql

`SELECT my_procedure(100) FROM dual; --` ❌ *Not allowed*

♦ Q5. Can procedures be overloaded in PL/SQL?

Answer:

✅ Yes, PL/SQL supports **overloading** of procedures (and functions) in **packages**, meaning:

- You can define multiple procedures with the same name but different parameter lists.

Allowed only in **packages**, not in standalone procedures.

♦ Q6. Can you **COMMIT** inside a procedure? Should you?

Answer:

✅ You can, but **!** you should avoid it in most cases.

- Committing inside procedures makes transactions **hard to control**, especially when the procedure is part of a larger process.
- It breaks the principle of **atomic transactions**.

Best practice: Let the calling program control `COMMIT` or `ROLLBACK`.

♦ Q7. Can a procedure return multiple values?

Answer:

✓ Yes, but not using `RETURN` like a function.

- Instead, procedures return multiple values via `OUT` or `IN OUT` parameters.

Example:

sql

```
PROCEDURE get_details(emp_id IN NUMBER, name OUT VARCHAR2, sal OUT NUMBER)
```

♦ Q8. Can a procedure be defined inside another procedure?

Answer:

✓ Yes, PL/SQL allows **nested procedure definitions**.

- A procedure can be declared and defined inside the `DECLARE` section of another procedure.
- The inner procedure is only visible to the outer one (not globally).

♦ Q9. What happens if you declare a procedure inside an anonymous block?

Answer:

✓ You **can** declare a procedure inside an anonymous block.

But it's **only valid** inside that block — it is **not stored in the database**.

Useful for temporary or one-time logic.

♦ Q10. Can a procedure be used in a trigger?

Answer:

✓ Yes, a procedure can be called from a trigger.

Best practice: Keep the trigger logic short and call a stored procedure from it to separate concerns.

Let me know if you want a quiz format or procedural questions based on real-world scenarios (like payroll, bank apps, etc.)!